BitBallot: Atomic Display Integrity for Verifiable Public Elections at Scale

BitBallot Author*

Independent Research Initiative

December 19, 2025

Abstract

We present **BitBallot**, a voting system designed for legally binding public elections under explicit institutional and physical deployment assumptions. Rather than attempting to replace democratic governance with cryptography alone, BitBallot separates cryptographic enforcement from institutional responsibility, assigning each its appropriate role.

At the protocol level, BitBallot leverages an execution-agnostic Layer-1 blockchain for ordering and finality, while enforcing correctness through verifiable programs and zero-knowledge proofs. We introduce *Atomic Display Integrity (ADI)*, a protocol-level security property that cryptographically binds voter intent, interface display, and recorded ballots into a single atomic authorization event, even under re-voting semantics.

BitBallot supports safe re-voting with last-vote-valid semantics and adopts a terminal-complete zero-knowledge tallying model that prevents intermediate information leakage prior to election closure. The system achieves public verifiability, privacy, and scalability without trusted execution environments, smart contracts, or dedicated Layer-2 infrastructure, enabling deployment on commodity hardware.

Together, these contributions demonstrate that verifiable, large-scale public elections can be realized using cryptographically rigorous yet institutionally realistic designs. Rather than attempting to replace democratic institutions with cryptographic mechanisms, BitBallot is designed to reinforce existing governance structures by making their operation more transparent, auditable, and accountable. Cryptography is used not as a substitute for institutional authority, but as a means to correct power imbalances by separating responsibilities that are traditionally conflated.

In particular, BitBallot distinguishes between cryptographic enforcement of vote integrity and institutional enforcement of eligibility, coercion prevention, and legal accountability. This separation of concerns avoids overloading cryptographic assumptions while enabling precise attribution of failures and responsibilities within the electoral process.

1 Introduction

Electronic voting for public elections must reconcile privacy, verifiability, scalability, and resistance to manipulation under realistic institutional constraints. Many existing systems either rely on trusted execution and opaque administration, or attempt to eliminate institutional trust entirely through cryptography, often adopting threat models that are incompatible with real-world democratic processes.

BitBallot is motivated by the observation that cryptography is most effective when it enforces correctness and transparency *within* clearly defined legal and physical deployment assumptions, rather than attempting to replace democratic institutions outright. The system is designed for supervised, legally binding elections and explicitly excludes unsupervised remote voting scenarios.

Contributions. This paper makes the following contributions:

- We introduce **Atomic Display Integrity (ADI)**, a novel protocol-level security property that cryptographically binds voter intent, interface display, and recorded ciphertext into a single atomic authorization event, even under re-voting semantics.
- We present a **terminal-complete zero-knowledge tallying** model that prevents any intermediate disclosure of election results while preserving full public verifiability of the final outcome.
- We demonstrate that **execution-agnostic Layer-1 blockchains**, when combined with verifiable program (vProg) execution, can support national-scale elections without smart contracts, centralized sequencers, or dedicated Layer-2 systems.

2 System Model and Threat Assumptions

BitBallot targets legally binding public elections conducted in supervised physical environments. The system explicitly separates threats addressable by cryptography from those requiring institutional, legal, or physical enforcement.

2.1 Participants

The protocol involves voters, election authorities, a set of guardians responsible for threshold decryption, and a public consensus layer providing ordering and finality. Voting terminals are provisioned, certified, and deployed by electoral authorities.

2.2 Adversary Model

We assume a computationally bounded adversary capable of network-level attacks, transaction censorship attempts, client compromise, and partial collusion among election officials. The adversary may observe all public blockchain data.

Physical coercion, intimidation, or surveillance in unsupervised private environments is considered out of scope and explicitly excluded by the deployment model.

2.3 Trust Assumptions

The consensus layer is assumed to satisfy standard honest-majority security. At least a threshold of guardians is assumed to behave honestly. Voting terminals are not trusted for correctness, but are assumed to operate in supervised environments.

3 Background and Related Work

End-to-end verifiable (E2E) voting has been extensively studied as a means to provide strong integrity and verifiability guarantees without trusting election authorities. Systems such as Helios, Prêt à Voter, Scantegrity, and their variants combine encrypted ballots, public bulletin boards, and voter-verifiable receipts to achieve cast-as-intended, recorded-as-cast, and tallied-as-recorded properties.

While these systems establish important cryptographic foundations, their design assumptions and security guarantees exhibit structural limitations when applied to large-scale, legally binding public elections.

3.1 Non-Atomic Separation of Intent, Display, and Authorization

Most E2E voting systems implicitly assume that voter intent, interface display, and ballot authorization occur coherently. However, this assumption is not enforced at the protocol level. In practice, voter confirmation is treated as a user-interface action, while authorization and submission are handled asynchronously by the client.

This separation introduces an attack surface in which compromised terminals can alter ballots between display and recording without immediate detection. Receipt-based verification and optional audits provide only post-hoc assurance and do not guarantee that the action authorized by the voter corresponds exactly to what was displayed at the moment of confirmation.

BitBallot addresses this gap by introducing *Atomic Display Integrity*, which enforces the atomicity of display, intent, and authorization as a single cryptographic protocol event.

3.2 Ambiguity Under Re-Voting Semantics

Several E2E voting systems prohibit re-voting altogether, while others support it through revocation lists or ad hoc supersession mechanisms. When re-voting is allowed, cast-as-intended guarantees typically apply only to individual submissions, leaving ambiguity about which voter action is authoritative in the final tally.

Such ambiguity complicates both voter assurance and formal security reasoning. BitBallot treats re-voting as a first-class operation. Atomic Display Integrity is enforced independently for

each submission, and last-vote-valid semantics deterministically select a single effective ballot per voter without weakening voter guarantees.

3.3 Intermediate Information Leakage

Many deployed E2E systems reveal partial information during the voting process, including bulletin board growth patterns, ciphertext aggregates, or early decryption artifacts. Even when individual ballots remain private, such leakage can influence voter behavior and undermine election fairness.

BitBallot adopts a terminal-complete zero-knowledge tallying model in which no meaningful information about vote distribution is revealed prior to the formal closure of voting, while preserving full public verifiability afterward.

3.4 Reliance on Trusted Auxiliary Infrastructure

E2E voting systems frequently depend on trusted bulletin boards, specialized hardware, or procedural controls to achieve their guarantees. These components introduce additional trust assumptions that are difficult to reconcile with adversarial environments or decentralized verification.

By contrast, BitBallot treats execution as untrusted but verifiable, relying solely on publicly verifiable proofs and a neutral consensus layer for ordering and finality. This design minimizes auxiliary trust assumptions while maintaining transparency and auditability.

4 Design Overview

BitBallot separates vote inclusion and finality from vote correctness. The underlying Layer-1 blockchain is used solely for immutable ordering and data availability, while all voting logic is enforced through verifiable off-chain computation. The consensus layer is agnostic to voting semantics and treats all submissions as opaque data, enabling independent verification without reliance on on-chain execution.

Each vote submission consists of an encrypted ballot, an atomic display commitment, and a zero-knowledge proof attesting to their mutual consistency. These artifacts are recorded on-chain without interpretation, while verification is performed independently by any observer. This architecture avoids smart contracts, trusted execution environments, and centralized sequencers, reducing attack surface and operational complexity.

4.1 Dummy Ballot Assignment

Each voting batch contains multiple ballots, all of which encode explicit candidate selections. Exactly one ballot reflects the voter's true intent, while the remaining ballots encode alternative candidate selections that are indistinguishable at the protocol level. This ensures that no external party can determine which selection corresponds to the voter's actual preference, even under coercion. All ballots in a voting batch encode syntactically valid candidate selections and satisfy

identical verification conditions at submission time. Dummy ballots are not invalid or placeholder votes. Instead, each dummy ballot encodes a concrete candidate selection, chosen such that all ballots in the batch remain equally plausible as the voter's true intent. This property is essential for coercion resistance: even if a voter is forced to reveal some ballot data, they cannot prove which selection represents their actual vote.

BitBallot does not require dummy ballots to cover all candidates, nor to avoid collisions by construction. While the tallying procedure ultimately selects at most one ballot per voter, the role of dummy ballots is to ensure that it remains computationally infeasible to infer which ballot will be counted based on transaction ordering, block height, or sparse network conditions. Uniform random assignment achieves this objective without introducing deterministic patterns or additional trust assumptions.

4.2 Aggregated Ballot Submission

All ballots generated by a single voter during a voting session, including the intended ballot and any dummy ballots, are processed locally as a batch. Rather than submitting each ballot as a separate transaction, the voter generates a single aggregated zero-knowledge proof attesting to the correctness of the entire ballot set with respect to protocol rules.

Only this aggregated proof and the associated opaque data are submitted to the mempool as a single transaction. This design eliminates observable differences in transaction count, timing, or structure that could otherwise serve as side channels for coercion or traffic analysis. From the perspective of the consensus layer, each voter contributes exactly one indistinguishable transaction.

4.3 Dummy Ballots and STARK-Based Aggregated Proofs

In BitBallot, each voter locally constructs a batch of ballots consisting of exactly one intended ballot and a bounded number of additional ballots that are indistinguishable in form and validity. Although the tallying procedure ultimately selects at most one ballot per voter, all ballots remain indistinguishable with respect to which one will be counted until tally time.

All ballots within a batch are verified through a single aggregated zero-knowledge proof instantiated as a STARK. This choice eliminates the need for trusted setup and ensures that all correctness guarantees remain publicly verifiable without reliance on any trusted party. From the perspective of the consensus layer, each voter submits exactly one indistinguishable transaction to the mempool, while all ballot-level semantics—including dummy structure and candidate selection—remain cryptographically hidden. This combination provides practical coercion resistance, robustness against side-channel inference, and long-term auditability suitable for legally binding public elections.

4.4 Verification Phases and Tally-Time Enforcement

Importantly, zero-knowledge proof verification is decoupled from the vote casting path. During voting, the Layer-1 consensus layer is responsible only for data availability and immutable ordering, and does not require full verification of STARK proofs. This design enables high-throughput vote submission without imposing cryptographic verification costs on the consensus layer.

Full verification of STARK proofs is performed during the tally phase, where latency constraints are relaxed and verification can be executed off-chain, in parallel, and over extended time windows. This separation ensures that national-scale elections remain practical while preserving strong post-election verifiability.

5 Voting Protocol

The voting protocol proceeds in three phases: setup, voting, and closure. During the setup phase, guardians jointly execute a standard threshold distributed key generation (DKG) protocol to derive the election public key. The protocol assumes no trusted dealer and ensures that no single guardian ever learns the full secret key.

Invalid or missing key shares are publicly detectable and treated as non-participation, consistent with the protocol's handling of guardian availability during tallying. Key generation is performed once per election and is not on the critical voting path, ensuring that setup costs do not affect voting throughput or availability.

5.1 Protocol Overview

During setup, election parameters, cryptographic keys, and guardian commitments are published. During the voting phase, voters may submit and resubmit ballots until the terminal block height is reached. Each submission includes an encrypted ballot, an atomic display commitment, and a zero-knowledge proof attesting to their consistency.

The protocol enforces last-vote-valid semantics by deterministically selecting the most recent valid submission per voter at tally time.

5.2 Re-voting and Failure Handling

BitBallot distinguishes between retransmission and re-voting. The 10-second retransmission mechanism is strictly limited to resubmission of an identical ballot commitment and proof, and is intended solely to mitigate transient network failures. Re-voting is explicitly defined as the submission of a new commitment, which is subject to standard rate limits and last-vote-valid semantics.

Because retransmissions are bitwise identical, they do not introduce additional information for traffic analysis, nor do they amplify DoS risk beyond that of a single submission. The consensus layer treats duplicate submissions idempotently.

6 Terminal-Complete Zero-Knowledge Tallying

BitBallot performs all tallying operations only after the voting phase has formally closed. Guardians collectively generate zero-knowledge proofs that the final tally is computed correctly from the set of effective ballots.

No intermediate decryptions or partial aggregates are revealed prior to closure. This terminal-complete approach prevents information leakage and strategic behavior during the election while preserving full public verifiability of the final result.

7 Atomic Display Integrity

End-to-end verifiable voting systems typically assume that the ballot presented for cryptographic authorization faithfully reflects what is displayed to the voter. In practice, this assumption may fail under compromised terminals or malicious user interfaces, leading to a discrepancy between the voter-observed selection and the cryptographically authorized ballot. Atomic Display Integrity (ADI) is introduced to eliminate this class of display-substitution attacks.

7.1 Definition

Atomic Display Integrity is a protocol-level property requiring that the act of voter authorization be cryptographically bound to the exact candidate selection displayed to the voter at the moment of confirmation. Formally, a ballot authorization is considered valid only if the encrypted ballot and its associated commitment are derived from the same display state that was presented to and acknowledged by the voter.

This binding ensures cast-as-displayed guarantees at the time of vote submission, preventing a terminal from displaying one selection while authorizing another without detection. [1]

7.2 Threat Model and Trust Boundary

Atomic Display Integrity assumes that the display-to-commitment conversion is implemented within a minimally trusted user interface boundary. ADI does not attempt to survive arbitrary compromise of the entire voting terminal. Instead, it ensures that any deviation between displayed content and cryptographic authorization becomes detectable and attributable.

In practice, the trusted computing base for ADI is limited to the display rendering path and the commitment generation logic. If this component is compromised, ADI fails closed by making the attack indistinguishable from a visibly malfunctioning terminal. Such failures are addressed through institutional safeguards and supervised deployment, rather than cryptographic recovery.

[1] Here, "indistinguishable" does not mean that correct and incorrect behavior are observationally identical to an external auditor. Rather, it means that an attacker cannot cause an unauthorized ballot to be accepted as a valid vote. Any deviation between displayed intent and cryptographic authorization results in a state equivalent to a terminal malfunction or protocol error, which is rejected by the system and addressed through institutional safeguards.

Importantly, Atomic Display Integrity does not address coercion through physical or social force. Preventing forced authorization requires institutional and physical protections, such as supervised polling stations and private voting booths, which are treated as explicit deployment assumptions elsewhere in this work.

7.3 Protocol Enforcement

ADI is enforced by requiring the voter's authorization to cover both the encrypted ballot and a display commitment representing the human-readable candidate selection. Any observer can verify that the submitted proof attests to the consistency of these artifacts without learning the underlying vote.

Because the authorization and commitment are generated atomically, the protocol prevents post-hoc modification of either component. A ballot that does not satisfy this consistency requirement is rejected during verification and excluded from tally consideration.

7.4 Security Implications

Atomic Display Integrity eliminates an entire class of attacks in which malicious software alters vote intent at the user interface layer while preserving syntactic correctness of the encrypted ballot. Unlike post-election verification mechanisms, ADI enforces correctness at the moment of vote casting.

By making the display state part of the cryptographically authorized payload, ADI strengthens cast-as-intended guarantees without expanding the trusted computing base beyond what is unavoidable in any interactive voting system. The property is compatible with re-voting semantics and aggregated ballot submission, and does not rely on trusted execution environments or specialized hardware.

8 Security Analysis

BitBallot ensures ballot privacy, integrity, and verifiability under standard cryptographic assumptions.

Atomic Display Integrity enforces cast-as-intended and recorded-as-displayed guarantees, even in the presence of compromised voting terminals. Re-voting combined with last-vote-valid semantics reduces the effectiveness of vote buying and coercion.

Ballot secrecy is preserved through encryption and threshold decryption, while public verifiability is achieved through zero-knowledge proofs.

8.1 Coercion Resistance and Indistinguishable Physical Receipts

BitBallot optionally allows terminals to produce a physical receipt corresponding to any ballot within a voter's submission batch. Importantly, such receipts are deliberately designed to be

cryptographically indistinguishable with respect to whether they correspond to the effective ballot or a dummy ballot.

As a result, receipts do not constitute transferable proof of voter intent. Any third party observing a receipt cannot determine whether it reflects the ballot ultimately counted in the tally. This property preserves coercion resistance even in the presence of physical artifacts, rendering vote-buying strategies economically irrational.

9 Scalability and Performance Evaluation

Because the consensus layer is used solely for ordering and data availability, BitBallot scales with the underlying transaction throughput.

Verification of zero-knowledge proofs and vProg execution is parallelizable and independent of consensus. This architecture supports national-scale elections within existing BlockDAG throughput limits.

9.1 Deployment Cost Considerations

BitBallot does not rely on trusted execution environments, specialized cryptographic hardware, or high-performance servers. All protocol operations, including ballot verification and zero-knowledge proof verification, are feasible on commodity hardware.

In practice, voting terminals can be implemented using repurposed or low-cost computing devices commonly available to electoral authorities, such as decommissioned office PCs. No persistent secret material is stored on voting terminals, allowing devices to be replaced or reused across election cycles without additional cryptographic risk.

This design lowers operational and deployment costs while preserving the security guarantees analyzed in this paper.

9.2 Rough Cost Estimate for STARK Verification

STARK proof verification is highly parallelizable and does not lie on the critical voting path. As a rough order-of-magnitude estimate, a single aggregated ballot proof can be verified in tens of milliseconds on commodity hardware using existing STARK implementations.

For a national-scale election with 10^8 voters, full verification can be distributed across independent verifiers and completed within practical time bounds (e.g., hours to days) using commodity server clusters. Importantly, full verification by every observer is not required for correctness; public availability of proofs enables selective verification and strong deterrence against manipulation.

10 Comparison with Existing Approaches

Compared to classical E2E voting systems, BitBallot uniquely enforces atomic authorization of voter intent and supports safe re-voting without trusted bulletin boards.

Unlike smart-contract-based voting systems, BitBallot avoids execution-layer trust assumptions and centralized sequencers, relying instead on execution-agnostic consensus and verifiable computation.

11 Deployment Assumptions and Institutional Enforcement

BitBallot is explicitly designed for legally binding public elections conducted under supervised and institutionally enforced conditions. Rather than attempting to resolve all failure modes cryptographically, the protocol makes its trust boundaries and non-cryptographic assumptions explicit. This section outlines the required deployment conditions and clarifies how cryptographic guarantees interact with legal and institutional enforcement.

11.1 Supervised Voting Environment

BitBallot assumes that voting is conducted in supervised physical environments, such as polling stations operated by electoral authorities. Voting terminals are provisioned, certified, and physically secured by the authority, including sealed enclosures and restricted peripheral access. Private or unsupervised voting environments, such as personal devices used at home, are explicitly out of scope.

These assumptions are necessary to address coercion through physical or social force. While the protocol provides cryptographic privacy and receipt-freeness properties, prevention of forced authorization relies on institutional safeguards such as private voting booths, supervision, and legal deterrence.

11.2 Guardian Roles and Threshold Assumptions

The protocol relies on a set of guardians responsible for threshold decryption and tally authorization. BitBallot assumes that at least a threshold t of guardians act honestly and remain available during the tally phase. Guardians operate independently and are institutionally accountable.

If fewer than t guardians participate, the protocol preserves vote secrecy but may fail to make progress. This reflects an explicit trade-off between liveness and legitimacy: BitBallot prioritizes preventing unauthorized tallying over forced completion in the presence of insufficient guardian participation.

11.3 Observable Guardian Participation

While intent cannot be inferred cryptographically, BitBallot provides objective, on-chain evidence of guardian participation. Each guardian is required to publish signed participation commitments within a predefined time window. Failure to do so is publicly observable and verifiable.

This mechanism allows legal and institutional processes to distinguish non-participation from protocol failure without relying on off-chain testimony. Intentional sabotage and accidental failure

remain legally distinct categories under law, but both result in identical cryptographic evidence of non-participation.

11.4 Election Invalidation and Recovery

If guardian participation falls below the required threshold and tally completion becomes impossible, the election is considered invalid under the protocol. Invalidation conditions are deterministic and publicly verifiable, preventing discretionary intervention by authorities.

Recovery procedures, including re-election or legal adjudication, are outside the scope of the protocol and are delegated to institutional processes. By making failure states explicit and observable, BitBallot avoids ambiguous outcomes while preserving democratic legitimacy.

11.5 Public Evidence of Guardian Participation

All key shares and decryption contributions in BitBallot are individually signed and attributable to specific guardians. As a result, both participation and non-participation are publicly verifiable, including which guardian contributions were used in the final tally.

This public attribution provides objective, cryptographic evidence suitable for legal and institutional procedures. While the protocol does not infer intent or assign blame, it establishes an immutable factual record of guardian behavior, enabling post-election accountability without reliance on off-chain testimony or discretionary interpretation.

11.6 Terminal Provisioning and Cost Assumptions

BitBallot is designed to operate on low-cost, commodity hardware provisioned by electoral authorities. All cryptographic operations, including zero-knowledge proof generation, are executed locally on the voting terminal without reliance on specialized hardware such as GPUs or trusted execution environments.

This design enables the use of refurbished or secondhand computing devices with minimal specifications, significantly reducing deployment costs while maintaining security guarantees. Cost optimization is treated as a first-class design constraint to ensure feasibility at national scale.

11.7 Separation of Cryptographic and Institutional Responsibility

A central design principle of BitBallot is the separation between cryptographic enforcement and institutional responsibility. Cryptography provides privacy, public verifiability, and objective evidence of protocol actions. Institutions provide coercion prevention, legal accountability, and adjudication of intent.

By explicitly defining this boundary, BitBallot avoids overextending cryptographic claims while enabling realistic deployment in existing democratic systems.

11.8 Voter and Terminal Authentication

BitBallot assumes the existence of an external voter eligibility and terminal registration process, enforced institutionally rather than cryptographically. Mechanisms such as resident registries, smart cards, NFC-based authentication, PIN codes, or soulbound credentials may be used to determine voter eligibility and terminal access, but are explicitly outside the scope of the protocol.

The security guarantees of BitBallot do not depend on the correctness of these mechanisms. Instead, the protocol ensures that once an authorized voter interacts with an authorized terminal, the resulting vote is cast, recorded, and tallied with integrity and public verifiability.

12 Conclusion

This paper introduced **BitBallot**, a voting system designed for legally binding public elections under explicit institutional and physical deployment assumptions. Rather than attempting to replace democratic governance with cryptography alone, BitBallot separates cryptographic enforcement from institutional responsibility, assigning each its appropriate role.

At the protocol level, BitBallot demonstrates that strong democratic guarantees can be achieved without trusted execution or application logic on the base layer. By leveraging an execution-agnostic BlockDAG Layer-1 and enforcing correctness through verifiable program execution and zero-knowledge proofs, the system achieves privacy, public verifiability, and scalability without introducing a dedicated Layer-2.

A central contribution of this work is **Atomic Display Integrity**, which ensures that voters cryptographically authorize exactly the ballot they intend at the moment of confirmation. Combined with last-vote-valid semantics and terminal-complete zero-knowledge tallying, BitBallot preserves voter assurance and privacy while enabling national-scale deployment.

By clearly defining what is enforced by protocol and what must be enforced by law, infrastructure, and governance, BitBallot avoids false promises while still expanding the design space of secure electronic elections.

A Reviewer Questions and Clarifications

This appendix addresses anticipated reviewer concerns by explicitly clarifying design choices that may appear unconventional when evaluated under common electronic voting assumptions.

A.1 Why does BitBallot exclude unsupervised remote voting?

Remote voting from private locations is intentionally excluded from the BitBallot threat model. While cryptographic mechanisms can mitigate certain digital attacks, they cannot reliably prevent physical coercion, surveillance, or intimidation in uncontrolled environments. Allowing unsupervised remote voting would therefore undermine the cast-as-intended guarantees provided by Atomic Display Integrity.

BitBallot prioritizes freedom from coercion over convenience. The protocol is designed for deployment in supervised voting environments where voters can safely express intent without external pressure.

A.2 Why are personal devices not supported?

Personal devices cannot be assumed to be free of malware, remote control software, or coercive monitoring. Even strong cryptographic protections cannot compensate for compromised endpoints.

BitBallot therefore assumes the use of certified voting devices provisioned and controlled by electoral authorities. This assumption ensures that Atomic Display Integrity is meaningful in practice rather than purely theoretical.

A.3 Why is guardian non-participation handled institutionally rather than cryptographically?

Guardians in BitBallot are publicly designated, legally accountable actors entrusted with threshold decryption responsibilities. Failure to submit decryption shares after the terminal condition constitutes deliberate obstruction of the democratic process.

Such behavior cannot be meaningfully mitigated through cryptography alone without introducing new trust assumptions or weakening finality. BitBallot therefore treats guardian non-participation as an institutional failure to be addressed through legal and constitutional enforcement, rather than as a protocol-level fault.

A.4 Why does BitBallot allow re-voting?

Re-voting is a deliberate design choice that improves voter assurance and resistance to coercion. By allowing voters to overwrite previous submissions until the closing block height, the protocol reduces the effectiveness of vote buying and intimidation.

Atomic Display Integrity ensures that each re-submission is independently authorized and verifiable, while last-vote-valid semantics guarantee a single effective ballot per voter in the final tally.

A.5 Why is a dedicated Layer-2 or rollup not used?

Layer-2 and rollup-based systems introduce additional trust assumptions, including sequencers, bridges, and governance mechanisms. These components complicate the security model and reintroduce centralized points of control.

BitBallot instead leverages an execution-agnostic Layer-1 for ordering and finality, while performing all computation off-chain via verifiable programs. This design preserves neutrality, minimizes trust, and avoids cross-layer dependencies.

A.6 Is Atomic Display Integrity merely a UI-level concept?

No. Atomic Display Integrity is a protocol-level security property enforced by cryptographic commitments and zero-knowledge proofs. The user interface is treated as untrusted, and any ballot not satisfying the atomic binding defined in Section 7 is rejected by the protocol.

ADI formalizes cast-as-intended and recorded-as-displayed guarantees under re-voting semantics, a setting not fully addressed by prior end-to-end verifiable voting systems.

A.7 Does BitBallot assume perfect availability of Kaspa Layer-1?

BitBallot assumes standard honest-majority security for the underlying consensus layer. Temporary censorship or network disruption may delay vote inclusion but does not compromise correctness or privacy. The last-vote-valid model allows voters to resubmit ballots if inclusion is temporarily prevented.